Paradigma Orientado a Objetos

Agenda:

- Modalidad de cursada – Condiciones de Aprobación
- Paradigmas de Programación
- Introducción a la POO – Conceptos
- El Lenguaje de Programación JAVA
- Breve explicación TP1
- Consultas

UADE

Paradigmas de Programación

Un **paradigma** es un enfoque o estilo que define la forma en que se diseñan y estructuran los programas. Es un conjunto de principios, técnicas y metodologías que guían la resolución de problemas mediante la programación.

Cada paradigma tiene reglas y estructuras, influenciando cómo se organizan los datos, cómo se escriben las instrucciones y cómo se ejecutan las operaciones.

Paradigmas de Programación

- **Paradigma imperativo:** se define claramente en el **código fuente** qué pasos debe ejecutar un programa y en qué secuencia
  - **Paradigma Procedural:** programación imperativa organizada mediante procedimientos, funciones o rutinas para evitar repetición de código y mejorar la claridad.
  - **Paradigma orientado a objetos**: Organiza el código en objetos que encapsulan datos y comportamientos. Se enfoca en el "cómo" resolver el problema mediante instrucciones detalladas que modifican el estado de los objetos
- **Paradigma Declarativo:** describe lo que debe solucionar el software (es decir, solo el resultado y no los pasos individuales).
  - **Paradigma funcional**: Se centra en funciones matemáticas y evita modificar el estado (ejemplo: Haskell, Lisp).
  - **Paradigma lógico**: Basado en reglas y hechos, donde la computación se realiza mediante inferencias lógicas (ejemplo: Prolog).

Objetivos de la Materia

- Conocer los conceptos básicos de la Programación Orientada a Objetos
- Pensar y diseñar en términos de Objetos
  - Representar problemas en términos de objetos que interactúan en relación cliente/servidor
  - Clasificar los conceptos de un problema, de acuerdo a distintas relaciones

¿Por qué Objetos?

Un nuevo enfoque para razonar sobre el Software

Una revolución Industrial (reutilización del software)

¿Que se busca?

- Incremento
  - Productividad
  - Calidad del software
  - Comprensión del software
  - Tiempo útil del software

Ciclo de Vida del Software

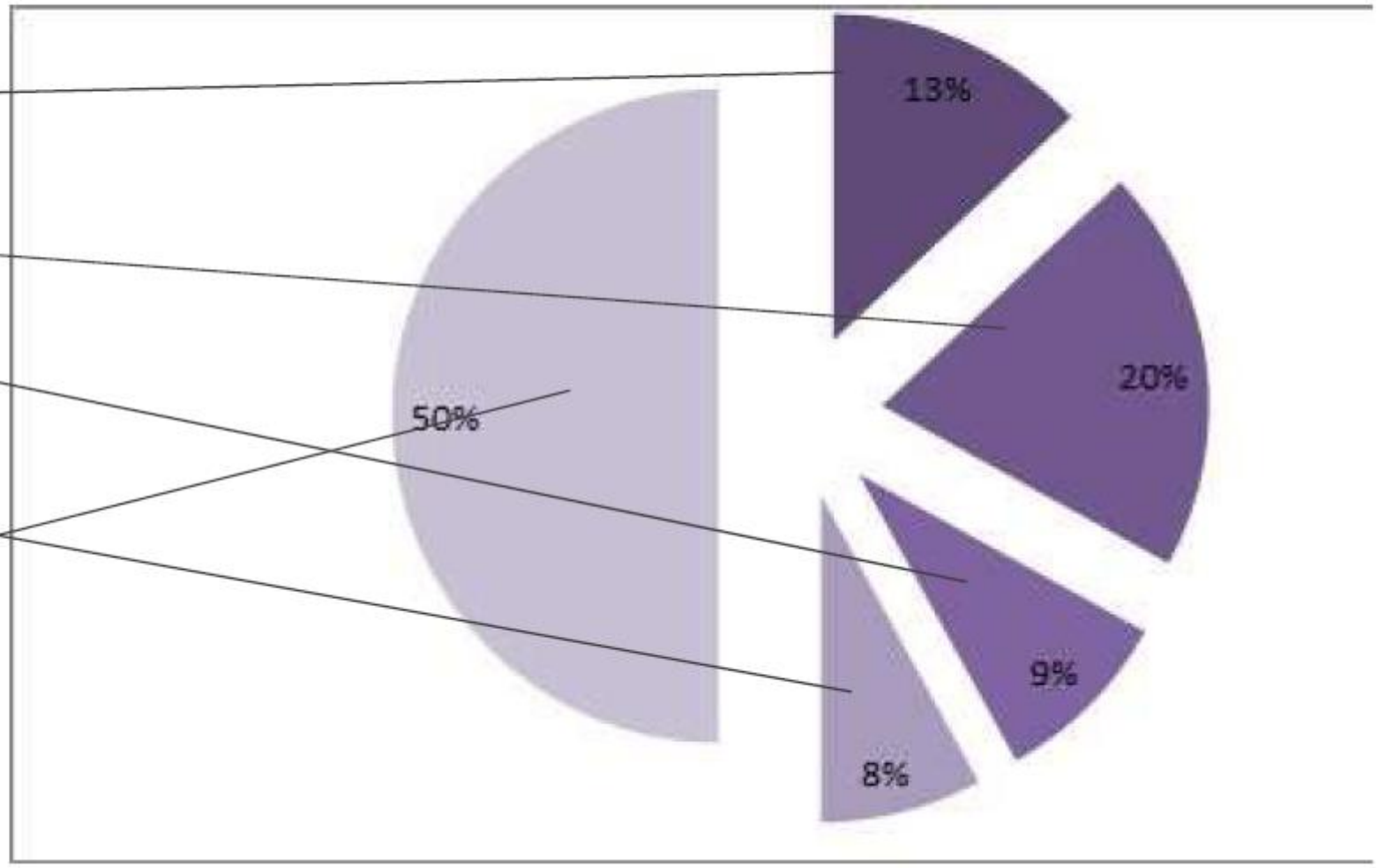
Análisis

Diseño

Implementación

Testing

Mantenimiento



Atributos de Calidad

- Mantenibilidad
- Extensibilidad
- Reusabilidad
- Compresibilidad

Aplicación Orientada a Objetos

Conjunto de **objetos** que interactúan mediante el **envío de mensajes** para cumplir un conjunto de **objetivos**

¿Qué es un Objeto?

- Un Componente de software (Único e irrepetible)
- Una entidad almacenada en memoria
- Un objeto encapsula datos y comportamiento en una unidad

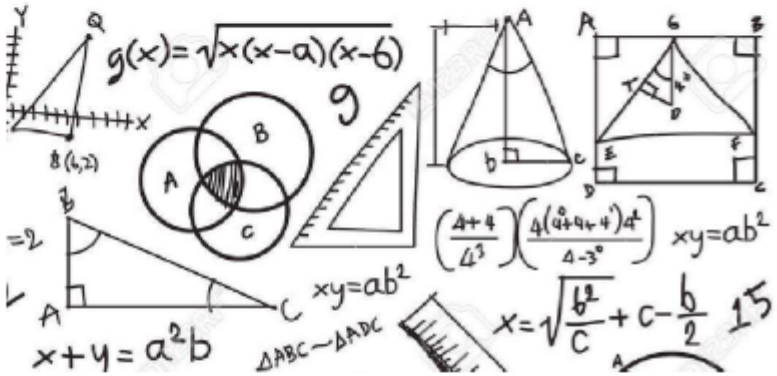
Objetos : Conceptos Reales

En un sistema de control de tráfico aéreo:
aviones, pistas, torres de control, etc.

En un sistema de alumnos de una Facultad:
Alumno, Curso, Docente, etc.

Objetos: Entidades Abstractas

- Una fórmula matemática



- Un evento (click del mouse)

Vista Dinámica de los Objetos

- Los objetos se crean y se destruyen en forma dinámica
- Los objetos tienen su propia identidad y encapsulan estado y comportamiento
- Las variables de instancia mantienen referencias a otros objetos Y las Variables de Clase ??
- El comportamiento de los objetos es definido por los métodos

Los Objetos

"Revelan qué pueden hacer y qué información pueden proporcionar, pero **no** revelan cómo lo hace o cómo lo conoce."

Encapsulamiento

- El encapsulamiento se refiere al ocultamiento de información o detalles
- Los objetos **encapsulan** sus **datos**
  - Los datos en los objetos son **privados**
  - Desde “el mundo exterior” no se puede **acceder** o **modificar** sus datos

Encapsulamiento

- El encapsulamiento se refiere al ocultamiento de información o detalles

- Los objetos **encapsulan** sus **datos**
  - Los datos en los objetos son **privados**
  - Desde “el mundo exterior” no se puede **acceder** o **modificar** sus datos
- Los **métodos** son (típicamente) **públicos**
  - Desde “el mundo exterior” se puede **enviar mensajes** que invoquen a los **métodos**

CLASE Vs INSTANCIA

- En un programa orientado a objetos generalmente se necesitan muchos objetos de un mismo tipo en un programa

Clase

---

molde para crear objetos con un determinado comportamiento y estado

Instancias

Clase

Clase Vs Instancia

CLASE

Un molde que define a las instancias

Un creador- una fábrica para crear objetos de un determinado tipo

INSTANCIA

Es instanciada (creada) por una clase

Ocupa espacio en memoria

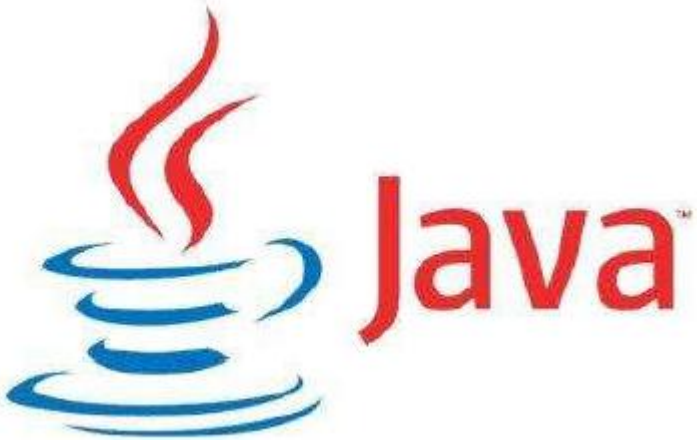
Mantiene un estado

Posee comportamiento

Conoce a qué clase pertenece

JAVA

El lenguaje **Java** es un lenguaje de **programación orientado a objetos**, que permite desarrollar aplicaciones para diferentes sistemas (es decir, aplicaciones **multiplataforma**)



JAVA

¿Utilizar el lenguaje Java **garantiza** que tengamos una solución correctamente orientada a objetos (con todas sus bondades)?

No, el lenguaje Java brinda al desarrollador cierta flexibilidad que puede ser usada de forma libre y, a veces, poco orientado a objetos

POO

¿Una solución orientada a objetos garantiza el éxito de la aplicación?

No, si ésta no cumple con los requisitos del cliente (por ejemplo, no es lo que el cliente pedía, es demasiado lenta, etc.)

Ejemplo: Avión

El avión posee información, por ejemplo capacidad, modelo, estado (aterrizando, despegando, en Vuelo, en tierra)

También tiene responsabilidades

Cuales? Qué otros atributos podemos definir?

Clase en Java

Primer paso, crear la clase

```
public class Avion {

....

}
```

Esto se guarda en un archivo **Avion.java**

Convenciones: Las clases siempre comienzan con mayúsculas

Clase en Java

Tenemos la clase, es decir tenemos el que crear la instancia.

```
Avion  air314 = new Avion()
```

Atributos

- Primero tenemos que establecer qué cosas va a tener nuestro avión: **Capacidad.** Un **estado.**
  - Podría tener más cosas: color, modelo, nombre, distancia recorrida, etc. Son cosas que tienen que ver con el Avión
- No tiene sentido poner un atributos que pertenecen otras entidades “precio” o “número de ticket de recital”
  Esto se suele denominar alta cohesión (los atributos del avión tienen relación con la entidad avión)

Clase Avion - Atributos

```
public class Avion {

   int capacidad;

   String estado;

   String color;

   Aeropuerto origen;

   Aeropuerto destino;

}
```

Clase Avión - Métodos

Supongamos que queremos cambiar el color del avión o preguntarle su color

```
public class Avion {
.... // los atributos anteriores
public void setColor(String unColor){
 color = unColor;
}
public String getColor () {
  return color;
}
```

Nota: Normalmente atributos y métodos sigue notación ***CamelCase***, pero comúnmente los métodos son verbos

Clase Avion - Método

Supongamos que queremos saber en que pista aterrizará el avión, pero para ello necesita preguntarle a la torre de control del aeropuerto destino

```
public Pista getPistaAterrizaje() {

      return destino.getTorreControl().getPista(this);

}
```

Definición de clases en Java- Métodos y atributos

¿hay algún orden entre ellos? No, los métodos, atributos, pueden ser declarados en cualquier orden
Entones...¿Es recomendable ponerlos en cualquier orden?

¡No! Por convención de código, se recomienda colocar atributos primero y métodos después.

Nota: Todos los desarrolladores Java suelen adherir a las convenciones, para facilitar compartir el código en equipos de desarrollo.

Constructor de un Objeto

El constructor puede verse como el método invocado para la construcción de un objeto. Cuando hacemos:

```
Avion a234 = new Avion()
```

Invocamos al constructor sin argumentos de la clase avión.

El constructor es quien se encarga de crear el objetos y setear los valores iniciales del mismo

Constructor de un Objeto JAVA

```
public class Avion {
   int capacidad;
   String estado;
   String color;

   public Avion(){   // CONSTRUCTOR SIN ARGUMENTOS
      capacidad = 100;   //VALORES POR DEFECTO
      estado = “en Tierra”;
      color = blanco;
   }
   public Avion (int cap, String col){//DOS PARAMETROS
      capacidad = cap;
      color = col;
      estado = “en Tierra”; //DEFECTO
  }
```

UN objeto puede tener múltiples constructores todos ellos deben tener una **signatura** diferente. Cuando se llama a un constructor es cuando se decide cual se invoca.

```
Avion a234 = new Avion(); //100, blanco

Avion a235 = new Avion(25, “Rojo”);
```